Plug and Play

Modular Semantics for Quantum Programming Languages

Jacob Trevor
j.trevor.1@research.gla.ac.uk

School of Computer Science, University of Glasgow, Glasgow

https://github.com/jaketrevor/qpl-semantics

A Tale of Two Semantics

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

- Apply the X gate
- To qubit 0
- Twice

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

$$\mathcal{G}_N[\![-]\!] : \ ? \to \mathbb{C}^{2^N \times 2^N}$$

$$\mathcal{G}_N[\![G\,\vec{q}]\!] = \pi_N^{-1}\vec{q} \cdot \overset{N}{\uparrow} G \cdot \pi_N \vec{q}$$

$$\mathcal{G}_N[\![a; b]\!] = \mathcal{G}_N[\![b]\!] \cdot \mathcal{G}_N[\![a]\!]$$

Gate Matrix (GM) Semantics

Consider the following program snippet:

```
X 0; X 0
```

What does this program *mean?*

$$\mathcal{G}_N[\![-]\!] : \ ? \to \mathbb{C}^{2^N \times 2^N}$$

$$\mathcal{G}_1[\![\texttt{X 0; X 0}]\!] = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Advantages of GM Semantics:

- Relatively simple to define (See above)
- Easy(ish) to explain
- Analytically, very nice
  - Full abstraction
  - Constructive, decidable equality
  - Directly compare matrices; 2^{2n} comparisons

In fact, lean can just do the check for you, if you set it up correctly:

```
53  example : [Gate 1 ℤ]Sem⟦ X0 ;;; X0 ⟧ = Sem⟦emptyProg⟧ := by
54    native_decide
```

A (slightly) more complicated program

```
X 0;
X 0;
Meas 0
```

- We can't give this a GM semantics - we need something *stronger*
- Give a semantics as a CPTP map
  - i.e. a function on density operators
  - Density operator = standard representation of quantum states
  - Which supports measurement

CPTP Map Semantics

$$\mathcal{D}_N[\![-]\!] : \ ? \to \rho_N \to \rho_N$$

$$\mathcal{D}_N[\![a; b]\!] = \mathcal{D}_N[\![b]\!] \circ \mathcal{D}_N[\![a]\!]$$

$$\mathcal{D}_N[\![G\,\vec{q}]\!] = \text{let } G' := \pi_N^{-1}\vec{q} \cdot \overset{N}{\uparrow} G \cdot \pi_N\vec{q}' \text{ in } \lambda\rho.G'\,\rho G'^{\dagger}$$

$$\mathcal{D}_N[\![\texttt{Meas q}]\!] = \lambda\rho. \sum_{i \in \text{ t, f}} P_i \rho P_i^{\dagger}$$

Comparison

CPTP Map semantics are a little less nice

- 2^{2n} vs 2^{4n} comparisons i.e. quadratic slowdown
- harder to mechanise

But!

- We can handle programs with measurement

Comparison

CPTP Map semantics are a little less nice

- 2^{2n} vs 2^{4n} comparisons i.e. quadratic slowdown
- harder to mechanise

But!

- We can handle programs with measurement

A tale as old as time: to handle more complicated languages, we need more complicated semantics

The baby and the bathwater

- Do we have to give up the nice analytic properties of our simple semantics to be able to work in more expressive languages?

The baby and the bathwater

- Do we have to give up the nice analytic properties of our simple semantics to be able to work in more expressive languages?

I say: No!

- Cut the language up into *features**
- Give each some semantics
- Define a way to *glue* features together
- In a way that lets us derive the semantics of the result

* we'll define this in a moment

Fragments and Feature sets

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires CPTP map ($\mathcal{D}$) semantics…

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires CPTP map ($\mathcal{D}$) semantics…
- but `X 0` can be given GM ($\mathcal{G}$) semantics

Language Features

Take a look at this program again:

```
X 0;
X 0;
Meas 0
```

Two distinct parts:

- `Meas 0` requires CPTP map ($\mathcal{D}$) semantics…
- but `X 0` can be given GM ($\mathcal{G}$) semantics

Idea: Cut this language up into “little languages”

Prior work

- What follows is essentially a specialisation of Wouter Swierstra's DTalC [1]
- Since I am mechanising it, a bit of extra trickery required to make the typechecker happy
  - i.e. containers
- This is a technique adapted from Jonas Maier's 2025 Masters Thesis (LRalC) [2]

Features

- A language *feature* is meant to capture “doing one kind of thing”
  - À la the little languages of Jon Bentley [3]

Features

- A language *feature* is meant to capture "doing one kind of thing"
  - À la the little languages of Jon Bentley [3]

So:

Unitary App (U) = { $G\ \vec{q}$ }
Effectful Measurement (M) = { Meas q }
Sequencing (Seq) = { _ ; _}

Features

- A language *feature* is meant to capture "doing one kind of thing"
  - À la the little languages of Jon Bentley [3]

So:

Unitary App (U) = { $G\ \vec{q}$ }
Effectful Measurement (M) = { Meas q }
Sequencing (Seq) = { _ ; _}

- Each feature becomes a *container*

Containers in 30 seconds

- Think of a container as consisting of:

1. A constant part
2. A number of holes

Containers in 30 seconds

- Think of a container as consisting of:

1. A constant part
2. A number of holes

- The extension of a container into Y tells you how to fill the holes with Ys

Containers in 30 seconds

- Think of a container as consisting of:

1. A constant part
2. A number of holes

- The extension of a container into Y tells you how to fill the holes with Ys
- We are interested in extending the container into more copies of itself…

Containers in 30 seconds

- Think of a container as consisting of:

1. A constant part
2. A number of holes

- The extension of a container into Y tells you how to fill the holes with Ys
- We are interested in extending the container into more copies of itself…
- Called the fixed point of a container

Constant Data

```
-- α the type of matrix entries
-- i the bitdwidth of the circuit

inductive uData i α  where
  | app : (h_size : n ≤ i) -- proof the gate fits
        → Gate n α         -- the gate to apply
        → PartialPerm n i  -- the list of targets
        → uData i α

inductive mData i (α : Type) where
  | meas : Fin i → mData i α
```

Containers

```
-- Command D n = "A command with constant data D and n holes"
abbrev Command (data : Type) (arity : Nat) : Container
  := {shape := data, pos := fun _ ⇒ Fin arity}


def U := Command uData 0
def M := Command mData 0

def Seq := Command Unit 2 -- sequence two commands
```

Key idea

- We can add feature containers together by taking their *coproduct* ($\oplus$)
  - The result is another container
- Our AST is now the *fixed point* of some container

Key idea

- We can add feature containers together by taking their *coproduct* ($\oplus$)
  - The result is another container
- Our AST is now the *fixed point* of some container

e.g. the AST of circuit, the language of our second example, can be defined as:

$$\text{Circuit} := (\text{Seq} \oplus U \oplus M).\text{fix}$$

- For more details, check slides from last week

Semantics = Algebra

Functors Refresher

- Say $\mathcal{C}$ is the category of types and total functions
- if $F : \mathcal{C} \to \mathcal{C}$ is an endofunctor on $\mathcal{C}$
- it's a map that takes types to types
  - $A : \text{Type} \vdash FA : \text{Type}$
  - Functors = type formers
- and functions to functions
  - $g : A \to B \vdash Fg : FA \to FB$

Functors generate syntax

- $F\,A$ = F-shaped expressions with As in the holes
- $F : \mathcal{C} \to \mathcal{C}$ is called the *signature* functor

Functors generate syntax

- FA = F-shaped expressions with As in the holes
- $F : \mathcal{C} \to \mathcal{C}$ is called the *signature* functor
- An algebra for the signature F (an F-algebra) is a pair:

$$(A : C, \alpha : FA \to A)$$

Functors generate syntax

- FA = F-shaped expressions with As in the holes
- $F : \mathcal{C} \to \mathcal{C}$ is called the *signature* functor
- An algebra for the signature F (an F-algebra) is a pair:

$$(A : C, \alpha : FA \to A)$$

- A is called the *carrier*

Functors generate syntax

- FA = F-shaped expressions with As in the holes
- $F : \mathcal{C} \to \mathcal{C}$ is called the *signature* functor
- An algebra for the signature F (an F-algebra) is a pair:

$$(A : C, \alpha : FA \to A)$$

- A is called the *carrier*
- α is called the *action*
  - α = "eval"
  - Given an expression (FA) compute it's value (A)

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$
  - Intuition: “recursive expressions”

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$
  - Intuition: “recursive expressions”
  - Expressions with more expressions in the holes

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$
  - Intuition: “recursive expressions”
  - Expressions with more expressions in the holes
  - There’s a function from $F(\mu F)$ to μF that *preserves all structure*
  - (i.e. it’s invertible)

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$
  - Intuition: “recursive expressions”
  - Expressions with more expressions in the holes
  - There’s a function from $F(\mu F)$ to μF that *preserves all structure*
  - (i.e. it’s invertible)
  - We call this function “in”

Fixed points

- μF = fixed point of F
  - $F(\mu F) \cong \mu F$
  - Intuition: "recursive expressions"
  - Expressions with more expressions in the holes
  - There's a function from $F(\mu F)$ to μF that *preserves all structure*
  - (i.e. it's invertible)
  - We call this function "in"
- Use this to define the initial algebra of F:

$$(\mu F : \mathcal{C},\ \text{in} : F(\mu F) \to \mu F)$$

Initial Algebra

$$(\mu F : \mathcal{C},\ \text{in} : F(\mu F) \to \mu F)$$

- **intuition:** “flatten out” expressions
- “Given expressions where the holes are filled with recursive expressions, give me a single recursive expression”

Initial Algebra

$$(\mu F : \mathcal{C},\ \text{in} : F(\mu F) \to \mu F)$$

- **intuition:** "flatten out" expressions
- "Given expressions where the holes are filled with recursive expressions, give me a single recursive expression"
- Quite boring: "eval" doesn't really do anything
  - This is by design
  - Instead, it specifies a way to deal with these "recursive expresions"
  - How to traverse μF

Recursion

- Initial algebra lets us define the fold function:

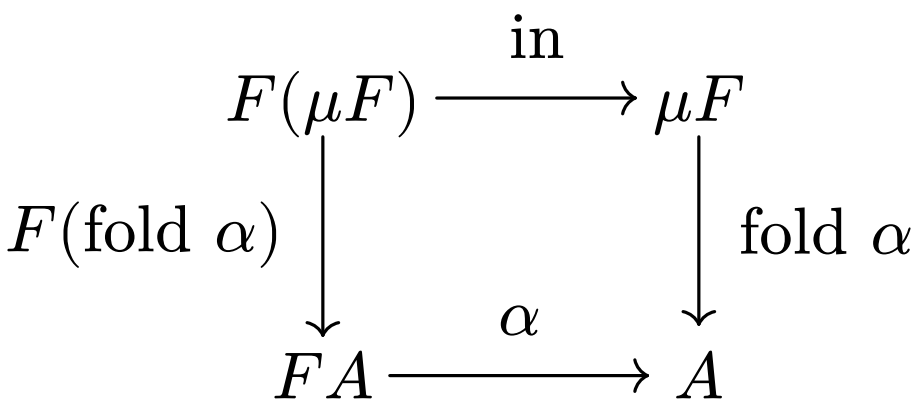
$$\text{fold} : (\alpha : FA \to A) \to (\mu F \to A)$$

Recursion

- Initial algebra lets us define the fold function:

$$\text{fold} : (\alpha : FA \to A) \to (\mu F \to A)$$

- It makes the following diagram commute:



When does this work?

- Not every functor has a well defined initial algebra

When does this work?

- Not every functor has a well defined initial algebra
- But those that are strictly positive always do

When does this work?

- Not every functor has a well defined initial algebra
- But those that are strictly positive always do
- This is the original motivation for containers:
  - Containers = data types with well defined recursion
  - We can define fold for containers

An intuition for fold_α

```
def fold (c : Container)
  (alpha : c.extension A → A)
  : c.fix → A
  | .mk s ch ⇒
    let ch' i := fold c alpha (ch i)
    alpha ⟨s, ch'⟩
```

An intuition for fold_α

```
def fold (c : Container)
  (alpha : c.extension A → A)
  : c.fix → A
  | .mk s ch ⇒
    let ch' i := fold c alpha (ch i)
    alpha ⟨s, ch'⟩
```

- Evaluate the children (recursively)

An intuition for fold_α

```
def fold (c : Container)
  (alpha : c.extension A → A)
  : c.fix → A
  | .mk s ch ⇒
    let ch' i := fold c alpha (ch i)
    alpha ⟨s, ch'⟩
```

- Evaluate the children (recursively)
- Then evaluate self
  - Hopefully at some point reaching a leaf with no children

An intuition for fold_α

```
def fold (c : Container)
  (alpha : c.extension A → A)
  : c.fix → A
  | .mk s ch ⇒
    let ch' i := fold c alpha (ch i)
    alpha ⟨s, ch'⟩
```

- Evaluate the children (recursively)
- Then evaluate self
  - Hopefully at some point reaching a leaf with no children

= Post order traversal

Semantics = Algebra

- (Collections of) features are containers A
- The AST of the resulting language is $A.\mathrm{fix}$

Semantics = Algebra

- (Collections of) features are containers A
- The AST of the resulting language is $A.\text{fix}$ $(= \mu(A.\text{ext}))$
  - (Recall: ext is a functor!)

Semantics = Algebra

- (Collections of) features are containers A
- The AST of the resulting language is $A.\mathrm{fix}$ $(= \mu(A.\mathrm{ext}))$
  - (Recall: ext is a functor!)
- A semantics for A in $\mathcal{X}$ is a function $\mathcal{X}[\![-]\!] : A.\mathrm{fix} \to \mathcal{X}$

Semantics = Algebra

- (Collections of) features are containers A
- The AST of the resulting language is $A.\text{fix}$ $(= \mu(A.\text{ext}))$
  - (Recall: ext is a functor!)
- A semantics for A in $\mathcal{X}$ is a function $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X}$
- As we saw, we can define this by giving $\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$
  - α is the action of an F-algebra

Semantics = Algebra

- (Collections of) features are containers A
- The AST of the resulting language is $A.\mathrm{fix}$ $(= \mu(A.\mathrm{ext}))$
  - (Recall: ext is a functor!)
- A semantics for A in $\mathcal{X}$ is a function $\mathcal{X}[\![-]\!] : A.\mathrm{fix} \to \mathcal{X}$
- As we saw, we can define this by giving $\alpha : A.\mathrm{ext}\ \mathcal{X} \to \mathcal{X}$
  - α is the action of an F-algebra

Semantics for A in $\mathcal{X}$ = Model of signature A in $\mathcal{X}$

Deriving Semantics

- If A and B have semantics in the same terms, then their coproduct does too

$$\frac{\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X} \qquad \beta : B.\text{ext}\ \mathcal{X} \to \mathcal{X}}{\alpha \otimes \beta : (A \oplus B).\text{ext}\ \mathcal{X} \to \mathcal{X}}$$

- Via pattern-matching on the shape, and applying the corresponding instance
- $\alpha \otimes \beta$ is a "coproduct model"
- See Lean for details

Subsumption

Subsumption

- Even with a finite collection of features, there's an infinite variety of feature sets
  - A, $A \oplus A$, $A \oplus A \oplus A$, ...
- It's also clear that A is somehow inside $A \oplus B$
  - e.g. {`X 0; X 0`} is inside {`X 0; X 0; M 0`}
- How do we formalise this notion?

Subsumption

A language *subsumes* another if there's a semantics preserving transformation from one to the other

$$A \preceq [\mathcal{X}]B \Leftrightarrow \exists(f : A \to B).\mathcal{X}[\![x]\!] = \mathcal{X}[\![f\,x]\!]$$

Subsumption

A language *subsumes* another if there's a semantics preserving transformation from one to the other

$$A \preceq [\mathcal{X}]B \Leftrightarrow \exists(f : A \to B).\mathcal{X}[\![x]\!] = \mathcal{X}[\![f\,x]\!]$$

When the semantics in question is obvious, I'll omit the $[\mathcal{X}]$ e.g.

$$A \preceq B$$

Isomorphisms

- If there's a semantics preserving function in both directions, then the languages can express *exactly the same things*
- i.e. they are isomorphic.
- We write

$$A \cong B \Leftrightarrow A \preceq B \wedge B \preceq A$$

Isomorphisms

- If there's a semantics preserving function in both directions, then the languages can express *exactly the same things*
- i.e. they are isomorphic.
- We write

$$A \cong B \Leftrightarrow A \preceq B \wedge B \preceq A$$

- If $A \preceq B$ but not $B \preceq A$ then we write $A \prec B$
- intuitively, B can express things A cannot

Normal form lemmas

$(1) A.\mathrm{fix} \preceq (A \oplus A).\mathrm{fix}$

$(2) (A \oplus A).\mathrm{fix} \preceq A.\mathrm{fix}$

$(3) (A \oplus B).\mathrm{fix} \preceq (B \oplus A).\mathrm{fix}$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

- (1) and (2) say $A \cong A \oplus A$
  - Need only consider A; $A \oplus A$ &c. are equivalent
- (3) gives us $A \oplus B \cong B \oplus A$

Normal form lemmas

$$(1) A \preceq A \oplus A \qquad (2) A \oplus A \preceq A$$

$$(3) A \oplus B \preceq B \oplus A$$

- (1) and (2) say $A \cong A \oplus A$
  - Need only consider A; $A \oplus A$ &c. are equivalent
- (3) gives us $A \oplus B \cong B \oplus A$
- Together, these give us a normal form for feature sets
  - Distinct features only
  - “Lexicographic” ordering

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set
- **Implication:** Programs in smaller feature sets are valid in larger ones
  - i.e. *syntactic subsumption*

Embedding lemma

$$A \preceq A \oplus B$$

- **Intuition:** There's an obvious way to embed A into a larger feature set
- **Implication:** Programs in smaller feature sets are valid in larger ones
  - i.e. *syntactic subsumption*
- **Reminder:** $\preceq$ means we have a SPF from A to $A \oplus B$
  - So for any program in A, we can generate the matching program in $A \oplus B$
  - e.g. $\text{emb}_{U \oplus M}\{-\}_U$ embed the program "$-$" in U into $U \oplus M$

Aside:

- I've omitted the proof for these, but there's a verified version in lean
- the proofs aren't particularly hard or interesting.

Summary so far:

- We can cut a language up into fragments
- We can add the fragments together as we wish
- So long as they all have semantics in the same terms, we can derive semantics the larger products

Summary so far:

- We can cut a language up into fragments
- We can add the fragments together as we wish
- So long as they all have semantics in the same terms, we can derive semantics the larger products

What if I don't want to give semantics in the same terms all the time?

Semantic strength and weakness

Semantic strength

Fact: If you give me a gate, I can give you a CPTP map:

$$e : \text{Gate } N \to \rho_N \to \rho_N$$

$$e \quad G \qquad\qquad = \lambda\rho.\ (G)\rho\left(G^\dagger\right)$$

- $\mathcal{D}_N$ is derivable from $\mathcal{G}_N$, so $\mathcal{G}_N$ is weaker than $\mathcal{D}_N$ ($\mathcal{G}_N \sqsubseteq \mathcal{D}_N$)
- Note $\mathcal{G}_N \sqsubseteq \mathcal{D}_N$ is the same way around as the type of e

Can we leverage this somehow?

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function
- We shouldn't lose information in the transformation

Aside: Sensible Functions

- cast needs to be a *sensible* choice of function
- We shouldn't lose information in the transformation
- Therefore, it should be an *embedding* i.e.

$$\exists p, p \circ e = \mathrm{id}$$

- Similar to the domain theory idea of *embedding/projection pair*
  - Thus the names e, p

Note that the same is not true in reverse:

- CPTP semantics ($\mathcal{D}_N$) can express more than GM semantics ($\mathcal{G}_N$)
- We can't embed $\mathcal{D}_N$ into $\mathcal{G}_N$
- Therefore $\mathcal{G}_N \sqsubset \mathcal{D}_N$

Strengthening

- If a language has a semantics in $\mathcal{X}$,
- and $\mathcal{X} \sqsubseteq \mathcal{Y}$,
- then we can use e to *derive* the semantics in $\mathcal{Y}$

$$\mathcal{Y}[\![-]\!] = e \circ \mathcal{X}[\![-]\!]$$

Strengthening

- If a language has a semantics in $\mathcal{X}$,
- and $\mathcal{X} \sqsubseteq \mathcal{Y}$,
- then we can use e to *derive* the semantics in $\mathcal{Y}$

$$\mathcal{Y}[\![-]\!] = e \circ \mathcal{X}[\![-]\!]$$

Prove things are equal in $\mathcal{X}$, by congruence also equal in $\mathcal{Y}$

Lift judgements from simple domains to more complex ones!

Strengthening: problem

Naïvely strengthening along $\mathcal{X} \sqsubseteq \mathcal{Y}$ yields:

$$\frac{\mathcal{X} \sqsubseteq \mathcal{Y} \qquad \alpha : A.\mathrm{ext}\ \mathcal{X} \to \mathcal{X}}{e \circ \alpha : A.\mathrm{ext}\ \mathcal{X} \to \mathcal{Y}}$$

- Which isn't quite right

Strengthening: problem

Naïvely strengthening along $\mathcal{X} \sqsubseteq \mathcal{Y}$ yields:

$$\frac{\mathcal{X} \sqsubseteq \mathcal{Y} \qquad \alpha : A.\mathrm{ext}\ \mathcal{X} \to \mathcal{X}}{e \circ \alpha : A.\mathrm{ext}\ \mathcal{X} \to \mathcal{Y}}$$

- Which isn't quite right
- remember: we need $A.\mathrm{ext}\ \mathcal{Y} \to \mathcal{Y}$ to define $A.\mathrm{fix} \to \mathcal{Y}$ via fold
  - So a model in $\mathcal{X}$ doesn't guarantee one in $\mathcal{Y}$
- Causes a problem when trying to lift judgements

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$
- Now consider that we want to rewrite along $\mathcal{X}[\![p]\!] = \mathcal{X}[\![q]\!]$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$
- Now consider that we want to rewrite along $\mathcal{X}[\![p]\!] = \mathcal{X}[\![q]\!]$
- ...in $A \oplus B$, where there is only a semantics in $\mathcal{Y}$, $\mathcal{X} \sqsubseteq \mathcal{Y}$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$
- Now consider that we want to rewrite along $\mathcal{X}[\![p]\!] = \mathcal{X}[\![q]\!]$
- ...in $A \oplus B$, where there is only a semantics in $\mathcal{Y}, \mathcal{X} \sqsubseteq \mathcal{Y}$
- The semantics in $A \oplus B$ is given by $\mathcal{Y}[\![-]\!] = \text{fold}\ (\beta_A \otimes \beta_B)$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$
- Now consider that we want to rewrite along $\mathcal{X}[\![p]\!] = \mathcal{X}[\![q]\!]$
- ...in $A \oplus B$, where there is only a semantics in $\mathcal{Y}, \mathcal{X} \sqsubseteq \mathcal{Y}$
- The semantics in $A \oplus B$ is given by $\mathcal{Y}[\![-]\!] = \text{fold}\ (\beta_A \otimes \beta_B)$
- Requiring us to have a model $\beta_A : A.\text{ext}\ \mathcal{Y} \to \mathcal{Y}$

Strengthening: problem (continued)

- Consider two programs $p, q : A.\text{fix}$
- A has a model in $\mathcal{X}$ ($\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$)
- Giving a semantics $\mathcal{X}[\![-]\!] : A.\text{fix} \to \mathcal{X} = \text{fold}\ \alpha$
- Now consider that we want to rewrite along $\mathcal{X}[\![p]\!] = \mathcal{X}[\![q]\!]$
- ...in $A \oplus B$, where there is only a semantics in $\mathcal{Y}, \mathcal{X} \sqsubseteq \mathcal{Y}$
- The semantics in $A \oplus B$ is given by $\mathcal{Y}[\![-]\!] = \text{fold}\ (\beta_A \otimes \beta_B)$
- Requiring us to have a model $\beta_A : A.\text{ext}\ \mathcal{Y} \to \mathcal{Y}$
- But for the congruence argument to work, we require:

$$\text{str}\ (\text{fold}\ \alpha) = \text{fold}\ \beta_A$$

Strengthening: fix

Split your features into:

- "terminal features" — with $n = 0$ holes
- "structural features" — with $n > 0$ holes,

Terminal features (#holes = 0)

- **Observation:** All extensions of a terminal features are isomorphic
  - extension = fill holes
  - no holes to fill $\rightarrow$ all fillings are the same

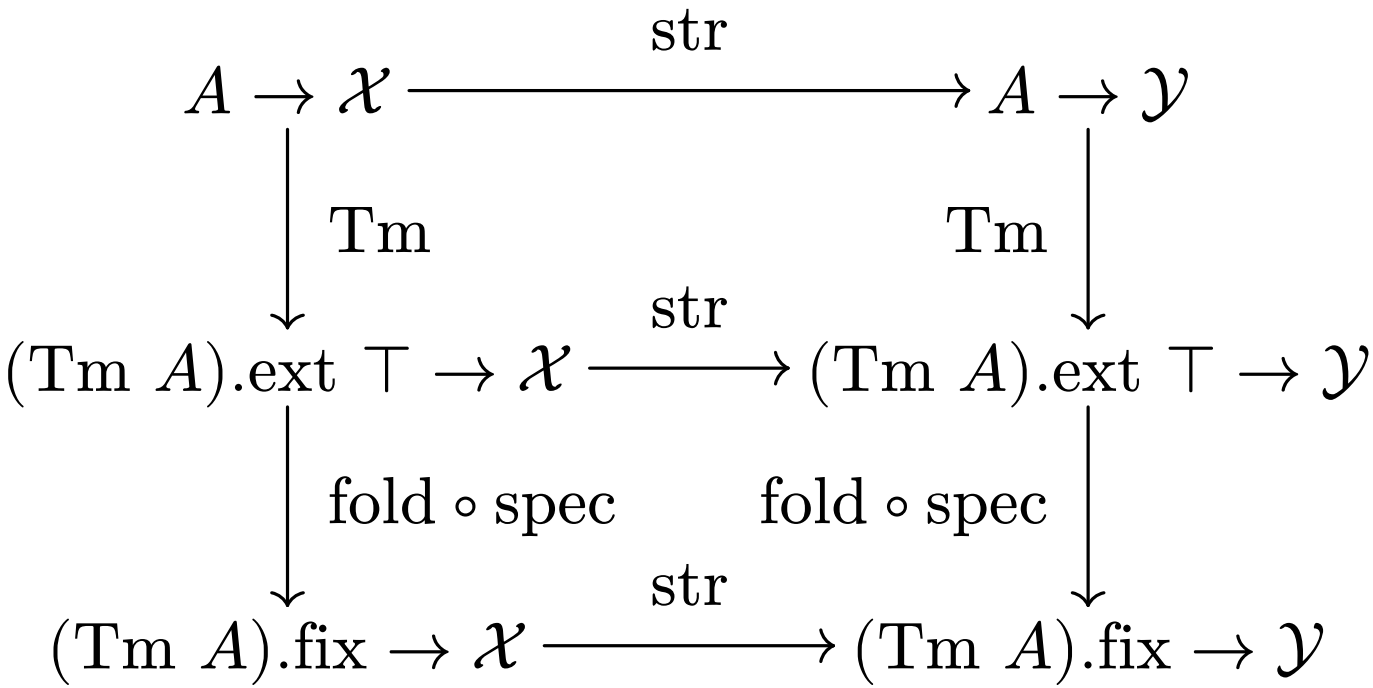
Terminal features (#holes = 0)

- **Observation:** All extensions of a terminal features are isomorphic
  - extension = fill holes
  - no holes to fill $\rightarrow$ all fillings are the same
- Define:

$$\text{spec} : \forall x\ y, (\text{Tm}\ A).\text{ext}\ x \rightarrow (\text{Tm}\ A).\text{ext}\ y$$

- (spec is self-inverse, and defines the isomorphism)
- It's enough to give semantics to a 'trivial filling' i.e. extension into Unit

Terminal Features

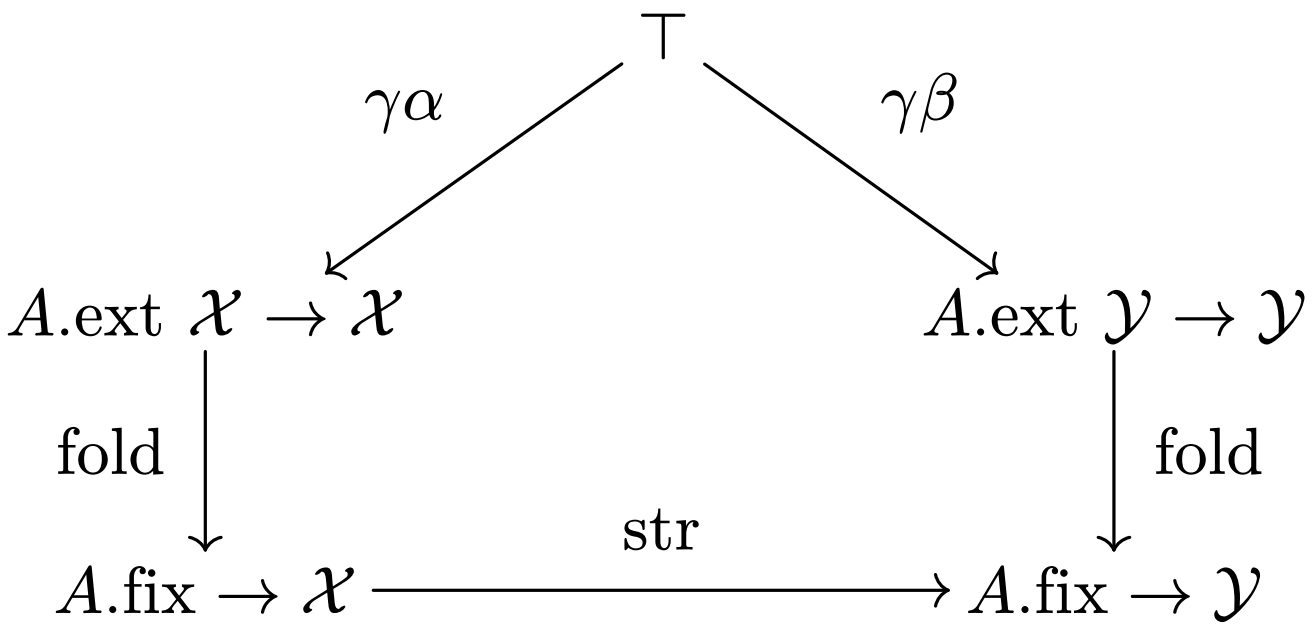


Structural features (#holes > 0)

- So far, I've found no way to automate the generation of semantics
- Instead, we require you to provide an algebraic model for each domain
- The two models must be "consistent" with the e/p pair
  - Strengthening from the weaker model should be the same as the stronger model

A diagram helps

- Two A-models $\alpha : A.\text{ext}\ \mathcal{X} \to \mathcal{X}$ and $\beta : A.\text{ext}\ \mathcal{Y} \to \mathcal{Y}$ are consistent ($\alpha \sim \beta$) iff this diagram commutes:



Current question: Coproduct coherence

Given

- two domains $\mathcal{X}, \mathcal{Y}, \mathcal{X} \sqsubseteq \mathcal{Y}$
- two features A, B,
- for each, a model in $\mathcal{X}$ (α_A, α_B) and $\mathcal{Y}$ (β_A, β_B)
- If their respective models are consistent wrt the embedding
  - $\alpha_A \sim \beta_A, \alpha_B \sim \beta_B$

Are the two resulting coproduct models consistent?

- $\alpha_A \otimes \alpha_B \sim \beta_A \otimes \beta_B$

Assuming that works out

Concretely

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side

Concretely

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$

Concretely

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$
- And I can then conclude that

$$\mathcal{D}_N[\![\mathrm{emb}_{U\oplus M}\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\mathrm{emb}_{U\oplus M}\{\}_U]\!]$$

Concretely

- If I know that $\mathcal{G}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{G}_N[\![\{\}_U]\!]$
  - where $\{\}$ is the empty program
  - prove easily by calculating the matrix representation of each side
- Then I know that $\mathcal{D}_N[\![\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\{\}_U]\!]$
- And I can then conclude that

$\mathcal{D}_N[\![\text{emb}_{U\oplus M}\{\texttt{X 0; X 0}\}_U]\!] = \mathcal{D}_N[\![\text{emb}_{U\oplus M}\{\}_U]\!]$

- And thus show that $\mathcal{D}_N[\![\{\texttt{X 0; X 0; M 0}\}_{U\oplus M}]\!] = \mathcal{D}_N[\![\{\texttt{M 0}\}_{U\oplus M}]\!]$
- Without ever having to look at a CPTP map!

Verifying certain kinds of program rewriting is now trivial!

Future work & Conclusions

Onwards

- A few definitions still use `sorry`

Onwards

- A few definitions still use `sorry`
- Next Stop: Hybrid languages
  - Semantics becomes much more interesting when classical co-processing is involved
  - Should be some nice strength & subsumption results to come
  - Dynamic qubit allocation is of particular interest

Applications

- Clear applications to *program optimisation*
  - Every equivalence between two programs induces a SPF
  - If this SPF is an “improvement”
  - It’s a verified optimisation
  - Valid even in more complex languages
  - See last slide of previous section for an example
  - Mechanise/automate the generation of such functions

Bibliography

[1] W. Swierstra, “Data types à la carte,” *J. Funct. Program.*, vol. 18, no. 4, pp. 423–436, 2008, doi: 10.1017/S0956796808006758.

[2] J. Maier, “Logical relations à la carte,” Master's thesis, 2026. [Online]. Available: https://ethz.ch/content/dam/ethz/special-interest/infk/inst-pls/plf-dam/documents/StudentProjects/MasterTheses/2025-Jonas-Thesis.pdf

[3] J. Bentley, “Programming pearls: little languages,” *Commun. ACM*, vol. 29, no. 8, pp. 711–721, Aug. 1986, doi: 10.1145/6424.315691.

That's all folks

Lean code repository:

https://github.com/jaketrevor/qpl-semantics

Contact me:

J.Trevor.1@research.gla.ac.uk

Any Questions?